Project Log

Sean

May 29, 2025

Introduction

This document contains the log of activities and hours spent on the individual project for the course individualProject for Avans.

In this course the student needs to make a project that incorporates the graphical card to do some calculations. The student needs to use OpenGL or OpenCL to do this.

Goal

The goal for this project is to use openCL to make a fluid simulation where water can be defined in a square space and the graphics card then calculates how the water flows. The focus first will be to make this in 2 dimensions. If this is done and there is time left the project will be expanded to 3 dimensions.

Log of Activities

1. **Read brightspace and init project**
    - **Activity** Read the brightspace course
    - **Activity** Setup code environment
    - **Activity** Setup git repository
    - **Activity** Write template for this log
2. **Google what is OpenCL**
    - **Activity** Read OpenCL landig page
    - **Activity** Read What is OpenCL
3. **Google openCL tutorial**
    - **Activity** Found getting started linux and followed part of tutorial with own knowledge to see if my envirement on new arch linux is working correctly.
    - **Activity** Installed openCL headers
    - **Activity** Followed Arch wiki to install openCL
    - **Note** First program did not work. Had to google around and after rereading Arch wiki found I also had to install opencl-cover-mesa package isntead of only openc-nvidia.
    - **Note** Added user to the video group using "sudo usermode -aG video sean", not sure if this was necessary
4. **Trying to use c++**
    - **Activity** Found OpenCL-CLHPP and looked at the example
    - **Founding** Saw I have to add extra find package to my cmakelists (OpenCLHeaders, OpenCLICDLoader and OpenCLHeaderCpp)
    - **Activity** After searching and trying things for about an hour an nothing working I remembered I can just add the raw hpp file to my project and use it that way.
5. **Search and follow basic tutorial of openCL**
    - **Activity** Found and followed Simple start with OpenCL and C++
    - **Note** In the tutorial a device is selected. At first there was no know device. After searching and testing for about 45 minutes, the problem was that I did not restart my computer.....
6. **Searching for fluids simulator c++ tutorials**
    - **Findings** Found two videos which might help me. But How DO Fluid Simulations Work? and Coding Adventure: Simulating Fluids

- **Findings** First thing in the second video is how to draw a circle. As I am programming in c++ first have to search how to actually show something on my screen.
- **note** Have not yet watched videos.

7. **Searching for c++ graphics libraries**
   - **Findings** Found multiple options for graphics. C# plus c++. SFML. Unity.
   - **Activities** Tried to setup c#, unity and SFML.
   - **Findings** C# kinda works. It was very annoying to setup on Linux. Have to learn to many things about c# to use it good.
   - **Findings** Unity does not work. It makes visualization very easy, but integration with c++ is very hard.
   - **Findings** SFML is a good option. It is a c++ library and is easy to use.
8. **Bouncing ball**
   - **Note** Decided to first make a bouncing ball in SFML to get a feel for the library.
   - **Activity** made a bouncing ball in SFML.
9. **Grid of bouncing balls**
   - **Note** Wanted to expend the previous code with multiple bouncing balls.
   - **Activity** made a grid of bouncing balls in SFML.
10. **Clean up code**
    - **Activity** Cleaned up the code of the grid of bouncing balls.
    - **Activity** Added a class for rendering.
    - **Activity** Added a class for the physics.
    - **Activity** Added submodules for SFML and ImuGui.
    - **Activity** Made code more modular.
11. **Made (not well implemented) equalizer**
    - **Activity** Made a equalizer with the balls so the balls/particles equalize over the screen.
    - **Note** The equalizer is not implemented very well. Code is a mess and physics are not that good.
12. **Improved physics**

- **Activity** Improved the physics of the simulation.
- **Activity** Added a class for a linear Quadtree.

13. **Added gravity**

    - **Activity** Added gravity to the simulation.
    - **Activity** Tweaked other settings to make the simulation more realistic.

14. **Rewrote physics to use GPU**

    - **Activity** Rewrote the physics to use the GPU.
    - **Note** Used OpenCL to do this.
    - **Note** Did not add mouse input yet.
    - **Note** Somehow the simulation is not working as expected. The particles are not moving as they should.

15. **Tried to implement better neighbor search**

    - **Activity** Tried to implement a better neighbor search using the morton codes.
    - **Note** I did not succeed in this. Particles are just disappearing.

Hours Spent

#	Activity	Hours
1	Read brightspace and init project	1
2	Google what is OpenCL	0.5
3	Google openCL tutorial	1
4	Trying to use c++	1.0
5	Search and follow basic tutorial of openCL	1.0
6	Searching for fluids simulator c++ tutorials	0.5
7	Searching for c++ graphics libraries	6
8	Bouncing ball	1.5
9	Grid of bouncing balls	1.5
10	Clean up code	3.5
11	went from falling balls to equalizer	5
12	Improve physics	8
13	Added gravity	6.5
14	Rewrote physics to use GPU	8
15	Tried to implement better neighbor search	4
	Total Hours	49

Results

The project is a 2D fluid simulation implemented using OpenCL for GPU acceleration and SFML for rendering. The current state of the simulation allows particles to be inserted into a square space and interact in a visually stable way, as long as gravity is not applied. Without gravity, the system behaves as expected and provides a basic demonstration of fluid-like movement. However, introducing gravity causes instability and breaks the simulation—particles begin to behave unpredictably, and the system no longer resembles a fluid. An attempt was made to improve performance and accuracy by implementing a more efficient neighbor search algorithm using Morton codes. Unfortunately, this approach was not successful and led to particles disappearing due to indexing issues or incorrect data structures. Despite these setbacks, the project has provided valuable insights into GPU programming, particle systems, and simulation techniques.

- Learned the basics of OpenCL.
- Understood how to use OpenCL for parallel computing.
- Learned how to use SFML for graphics.
- Learned how to use ImGui for GUI.
- implemented Quadtree for neighbor search.
- Implemented a simple fluid simulation.